

Midterm practice workshop

CSC103 Programming Fundamentals / Lecture 18

Dr. Muhammad Farhan

Associate Professor of Computer Science
Department of Computer Science
COMSATS University Islamabad
Sahiwal Campus

Fall 2026

The problem for this lecture

Write the full console version for a user-specified nonnegative count. Use a method `isPass(int mark)`.

Read count and initialise both accumulators.

Learning objectives

- Solve an integrated problem under time constraints
- Justify input and boundary cases
- Review solutions using an explicit rubric

CLO-1 and CLO-2
Practice task and assumptions
Decomposition
Boundary test design
Peer-review rubric

Practice task and assumptions

- Process a fixed number of valid marks in 0..100.
- Calculate the average and count marks at least 50.
- Handle zero records without division.

Example

Input example: 4 then 40, 50, 60, 70

Expected count passing: 3

Expected average: 55.0

Decomposition

- Separate reading marks from computing a classification.
- Choose a counter, sum and passing count.
- Define what should happen when the count is zero.

Example

State: `i, sum, passes, count`

Repeated step: `read one mark, update totals`

Final step: `report or print No data`

Boundary test design

- A mark of 49 fails the threshold and 50 passes.
- A count of zero exercises the no-data branch.
- Equal values make averages easy to verify manually.

Example

Case A: count 0

Case B: count 1, mark 50

Case C: count 2, marks 49 and 50

Peer-review rubric

- Specification and assumptions: 2 practice points.
- Correct control flow and calculation: 4 points.
- Boundary tests: 2 points. Clear explanation and style: 2 points.

Example

Reviewer: cite one concrete strength.

Reviewer: identify one reproducible failure.

Author: explain the correction.

A specification determines the loop state

- Reading a fixed number of marks requires a counter.
- The average requires a sum and a count.
- A passing count requires a Boolean rule applied to every mark.

Example

```
count = requested number of records  
sum = sum of accepted records  
passes = number at least 50
```

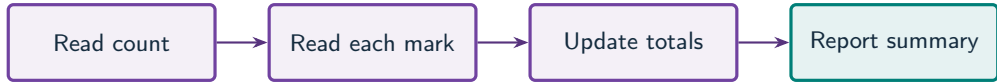
Zero records need a separate outcome

- The loop executes no bodies when the requested count is zero.
- A zero count cannot support an arithmetic mean.
- Printing No data states the situation without inventing a numeric result.

Example

```
Input: 0  
No mark tokens are read.  
Output: No data
```

An integrated marks problem has stages



What to notice

Separate the loop state from the final reporting decision.

Key distinctions

Case	Marks	Mean	Passing count
Ordinary	40,50,60,70	55.0	3
Threshold	50	50.0	1
Mixed boundary	49,50	49.5	1
No records	None	No data	0

These distinctions determine which operation or control structure fits the problem.

First example: execution

```
int sum = 0, passes = 0;
for (int mark = 40; mark <= 70; mark += 10) {
    sum += mark;
    if (mark >= 50) passes++;
}
System.out.println(sum / 4.0);
System.out.println(passes);
```

First example: result

55.0

3

The values are 40, 50, 60 and 70.

This deterministic demonstration substitutes for console input.

Boundary test design

Peer-review rubric

Guided practice

Write the full console version for a user-specified nonnegative count. Use a method `isPass(int mark)`.

Attempt the solution before continuing.
Use the definitions and examples from this lecture.

Solution strategy

- Read count and initialise both accumulators.
- Read count marks, adding to sum and incrementing passes when isPass returns true.
- Handle count zero separately and otherwise report the mean and passing count.

Sample console input
4 40 50 60 70

Complete solution (1/2)

```
public class Solution18 {  
    public static void main(String[] args) throws Exception {  
        java.util.Scanner in = new java.util.Scanner(System.in);  
        int count = in.nextInt();  
        int sum = 0;  
        int passes = 0;  
        for (int i = 0; i < count; i++) {  
            int mark = in.nextInt();  
            sum += mark;  
            if (isPass(mark)) passes++;  
        }  
    }  
}
```

Complete solution (2/2)

```
    if (count == 0) System.out.println("No data");
    else {
        System.out.println((double) sum / count);
        System.out.println(passes);
    }
}

static boolean isPass(int mark) {
    return mark >= 50;
}
}
```

Execution trace

Mark	sum after addition	Pass?	passes
40	40	No	0
50	90	Yes	1
60	150	Yes	2
70	220	Yes	3

Follow the changing state in execution order.

Solution result

55.0

3

Why this result follows
Handle count zero separately and
otherwise report the mean and
passing count.

A common incorrect approach

```
double average = sum / count;  
// count may be zero; sum and count are int
```

Example

First handle `count==0`. On the nonzero branch use `(double) sum / count` to avoid truncation.

Boundary and failure checks

Check the stated input assumptions.

Revise your workshop solution and attach three tests with expected and actual results.

Read count. Loop count times, reading a mark and accumulating sum and passes. If count>0 print (double)sum/count, otherwise No data. isPass returns mark>=50.

Check your understanding

What test separates ≥ 50 from > 50 ? What part of this solution is reusable without console input?

Write a prediction and explain the rule that supports it.

Answer and explanation

A mark exactly 50. The isPass calculation and, after further decomposition, the summary calculation.

First handle `count==0`. On the nonzero branch use `(double) sum / count` to avoid truncation.

Compare cases and predict outcomes

Input marks	Mean	Passes (≥ 50)
49, 50, 51	50.0	2
80	80.0	1
No records	No data	0

Discuss

Explain each expected result before running the example. Which case would reveal a wrong assumption?

Apply the idea

Independent practice

Design a three-case test plan for a console marks summary: an ordinary case, the pass boundary, and zero records. Explain the expected results before coding.

1. Work through the input by hand.
2. Write or correct the program.
3. Justify the expected result.

Solution and reasoning

Ordinary: 60 80 -> mean 70.0, passes 2
Boundary: 49 50 -> mean 49.5, passes 1
Zero records: count 0 -> No data

- Each expected result is derived independently of the program.
- Testing 49 and 50 detects an accidental strict $>$ comparison.
- Zero records must avoid division by zero.

Sorting three values with compare-and-swap

- Swap the first pair if out of order, then the second pair, then the first pair again.
- The third comparison repairs the first pair after the second swap.
- The method changes local parameter values and prints them; it does not mutate caller primitives.

Source: supplied ISB campus slides. See speaker notes and [ISB_Source_Map.md](#).

Worked problem: Sorting three values with compare-and-swap

Task

Adapt the source `displaySortedNumbers` method and trace inputs 9, 2 and 5.

Input: Values are fixed in the program for a reproducible demonstration.

Before running

Predict the output and identify the variables that change.

Complete ISB example (1/1)

```
public class IsbLecture18 {  
    public static void main(String[] args) throws Exception {  
        displaySortedNumbers(9, 2, 5);  
    }  
    static void displaySortedNumbers(double a, double b, double c) {  
        double temp;  
        if (a > b) { temp = a; a = b; b = temp; }  
        if (b > c) { temp = b; b = c; c = temp; }  
        if (a > b) { temp = a; a = b; b = temp; }  
        System.out.println(a + " " + b + " " + c);  
    }  
}
```

Worked trace and expected result

Stage	Values	Reason
Start	9, 2, 5	Unsorted
Compare first pair	2, 9, 5	Swap 9 and 2
Compare second pair	2, 5, 9	Swap 9 and 5
Compare first pair again	2, 5, 9	No swap

Expected console output

2.0 5.0 9.0

Extend the ISB example

Practice

Which input makes the final comparison necessary?

Explain your reasoning

For 5, 9, 2 the second swap gives 5, 2, 9, so the final swap is required.

Lecture summary

- an integrated problem under time constraints
- input and boundary cases
- solutions using an explicit rubric
- Read count and initialise both accumulators.
- Read count marks, adding to sum and incrementing passes when isPass returns true.
- Handle count zero separately and otherwise report the mean and passing count.

After-class practice

Revise your workshop solution and attach three tests with expected and actual results.

Syllabus reading

Lectures 1–16

Next lecture: Recursion and backtracking